Praktikum JavaScript & TypeScript

Soal Modul 6 - Modules dan Standard Library

1. Tugas

Kamu menulis modul `kasir.js` untuk mesin kasir supermarket. File utama `soal-js6.js` meng-import fungsi dari modul itu, lalu memanggilnya. Data `produk` ada di bagian `DATA` file utama.

id	nama	kategori	harga
K01	Beras 5kg	Sembako	78000
K02	Telur 1kg	Sembako	28000
K03	Kopi Sachet	Minuman	1500
K04	Sabun Batang	Perawatan	4500
K05	Teh Botol	Minuman	5000

Tabel 1: Data `produk`. Tipe: `id`, `nama`, `kategori` string; `harga` number.

Pembagian kerja dua file:

File	Isi
`Jawaban/kasir.js`	Semua fungsi yang di-`export`: Beginner 1–3, Intermediate 1–3, dan `buatStruk` (Advanced).
`Jawaban/soal-js6.js`	`import`, data, `cobaTambah` (Intermediate 2), `cetakStruk` (Advanced), jawaban **(jelaskan)**, dan `PEMANGGILAN`.

⚠ Perhatian

1. Salin `Soal/starter.js` ke `Jawaban/soal-js6.js`, dan `Soal/kasir.js` ke `Jawaban/kasir.js`. Kedua file harus berada di folder yang sama.
2. Lengkapi fungsi bertanda `TODO`. Jangan mengubah baris `import`, bagian `DATA`, dan `PEMANGGILAN`.
3. Jalankan `node Jawaban/soal-js6.js`. Kotak **Output dari** `console.log(...)` di bawah setiap tugas menunjukkan baris `console.log` yang mencetaknya dan output yang harus keluar persis.
4. Soal bertanda **(jelaskan)** dijawab sebagai komentar di `soal-js6.js`, di bawah baris `// JAWABAN ...`.
5. Semua number adalah bilangan bulat, kecuali disebut lain.

1.1. Beginner (0.3)

1. **Format rupiah** (di `kasir.js`).
   - Fungsi: `formatRupiah(n)`. Input: `n` number.
   - Output: string `Rp` disambung angka dengan pemisah ribuan titik, tanpa spasi.
   - Wajib memakai `Intl.NumberFormat` dengan locale `"id-ID"`.

   Output dari `console.log(formatRupiah(1250000))`

   ```
   Rp1.250.000
   ```

   Output dari `console.log(formatRupiah(500))`

   ```
   Rp500
   ```

2. **Pembulatan harga** (di `kasir.js`).

- Fungsi: `bulatkanKe(n, kelipatan)`. Input: `n` number, `kelipatan` number lebih dari 0.
- Output: number, kelipatan `kelipatan` terkecil yang tidak kurang dari `n`. Jika `n` sudah kelipatannya, kembalikan `n`.
- Wajib memakai method `Math`.

Output dari `console.log(bulatkanKe(12340, 500))`

```
12500
```

Output dari `console.log(bulatkanKe(12500, 500))`

```
12500
```

3. **Kategori unik** (di `kasir.js`).
   - Fungsi: `kategoriUnik(produk)`. Input: `produk` array `Produk`.
   - Output: array string, kategori tanpa duplikat, urut sesuai kemunculan pertama.
   - Wajib memakai `Set`, tanpa `for` dan tanpa `includes`.

Output dari `console.log(kategoriUnik(produk).join(", "))`

```
Sembako, Minuman, Perawatan
```

4. **(jelaskan)** Temanmu menulis baris ini di file utama, lalu menjalankannya. Coba di file terpisah dalam folder yang sama dengan `kasir.js`.

```
console.log("mulai");
import { formatRp } from "./kasir.js";
```

   - Tulis pesan error lengkapnya.
   - Mengapa `mulai` tidak tercetak, padahal `console.log` ditulis sebelum `import`?
   - Tulis baris `import` yang benar.

1.2. Intermediate (0.3)

1. **Index produk** (di `kasir.js`).
   - Fungsi: `buatIndex(produk)`. Input: `produk` array `Produk`.
   - Output: `Map` dengan key `id` dan value object produknya.
   - **(jelaskan)** Di Modul 3, produk dicari dengan `produk.find(...)`. Jika data berisi 10.000 produk dan keranjang berisi 50 item, berapa kira-kira perbandingan id yang dilakukan `find`, dibandingkan dengan `index.get(...)`? Mengapa `Map` lebih cepat?

Output dari `const index = buatIndex(produk);` lalu `console.log(index.size)`

```
5
```

Output dari `console.log(index.get("K03").nama)`

```
Kopi Sachet
```

Output dari `console.log(index.get("X99"))`

```
undefined
```

2. **Validasi jumlah barang.**
   - Di `kasir.js`: `validasiQty(qty)`. Input: `qty` nilai apa saja. Jika `qty` bukan bilangan bulat atau 0 atau negatif, **lempar** `Error` dengan pesan `qty harus bilangan bulat positif`. Jika valid, kembalikan `qty`.
   - Di `soal-js6.js`: `cobaTambah(qty)`. Panggil `validasiQty` di dalam `try...catch`. Output: `OK: <qty>` jika valid, atau `GAGAL: <pesanError>` jika tertangkap.
   - String `"2"` termasuk tidak valid, karena bukan number.

Output dari `console.log([3, 0, 2.5, "2"].map(cobaTambah).join("\n"))`

```
OK: 3
GAGAL: qty harus bilangan bulat positif
GAGAL: qty harus bilangan bulat positif
GAGAL: qty harus bilangan bulat positif
```

3. **Kode temanmu:** `formatTanggal(tgl)` (di `kasir.js`).
   - Output yang benar: string format `DD-MM-YYYY`. Hari dan bulan selalu 2 digit.
   - Masalah: `formatTanggal(new Date(2026, 10, 5))` menghasilkan `5-10-2026`.
   - **(jelaskan)** Mengapa bulannya `10`, padahal tanggal itu bulan November? Mengapa `new Date(2026, 10, 5)` ditulis dengan `10` untuk November? Lalu perbaiki.

Output dari `console.log(formatTanggal(new Date(2026, 10, 5)))`

```
05-11-2026
```

Output dari `console.log(formatTanggal(new Date(2026, 0, 31)))`

```
31-01-2026
```

1.3. Advanced (0.4)

Advanced dinilai dari **pendekatannya**. Bagian (a) wajib dan menentukan nilai. Bagian (b) opsional. Kode yang kosong tidak mengurangi nilai.

1. **Struk belanja.**
   - Di `kasir.js`: `buatStruk(keranjang, index, tanggal)`.
     - Input: `keranjang` array `{ id: string, qty: number }`, `index` hasil `buatIndex`, `tanggal` object `Date`.
     - Untuk setiap item, sesuai urutan: jika produk tidak ada di `index`, lempar `Error` dengan pesan `produk <id> tidak ditemukan`. Lalu validasi `qty` dengan `validasiQty`.
     - Output jika semua valid: string beberapa baris, dipisah `"\n"`:

       ```
       Struk <tanggalDD-MM-YYYY>
       <nama> x<qty> = <subtotalRupiah>      (satu baris per item)
       Total: <totalRupiah>
       ```

   - Di `soal-js6.js`: `cetakStruk(keranjang, index, tanggal)`. Panggil `buatStruk` di dalam `try...catch`. Jika tertangkap, kembalikan `GAGAL: <pesanError>`.

   a. **(jelaskan, wajib)** Tulis pendekatanmu di komentar `// JAWABAN Advanced 1a`. Tulis langkah-langkah `buatStruk`, fungsi dari Beginner dan Intermediate yang dipakai di setiap langkah, dan bentuk data di setiap langkah. Jelaskan juga: mengapa error **dilempar** di `kasir.js` tetapi **ditangkap** di `soal-js6.js`, bukan ditangkap di `kasir.js` sendiri?

   b. **(opsional)** Tulis kodenya.

Output dari `console.log(cetakStruk([{ id: "K01", qty: 1 }, { id: "K02", qty: 2 }, { id: "K03", qty: 10 }], index, tanggal))`

```
Struk 05-11-2026
Beras 5kg x1 = Rp78.000
Telur 1kg x2 = Rp56.000
Kopi Sachet x10 = Rp15.000
Total: Rp149.000
```

Output dari `console.log(cetakStruk([{ id: "K01", qty: 1 }, { id: "X99", qty: 1 }], index, tanggal))`

```
GAGAL: produk X99 tidak ditemukan
```

Output dari `console.log(cetakStruk([{ id: "K05", qty: 0 }], index, tanggal))`

```
GAGAL: qty harus bilangan bulat positif
```

2. Pengumpulan

⚠ Perhatian

- Kode soal: `Jawaban/soal-js6.js` dan `Jawaban/kasir.js`. Jawaban **(jelaskan)** ditulis sebagai komentar di `soal-js6.js`.
- Coba Sendiri dari HO dikerjakan di fungsi `cobaSendiri()` pada `soal-js6.js`. Jalankan dengan `node Jawaban/soal-js6.js coba`.
- Jalankan `npm run selesai -- --durasi "..." --macet "..."`, lalu minta asisten memeriksa.